VERA RBL-C64
FPGA Build & Programming Guide

iCE40UP5K – Rev 6

Commodore 64 expansion-port interface:
VERA registers in IO2 and GeoRAM 256 KB on external SRAM

Last updated: October 23, 2026

0 Contents

1 Overview **2**
1.1 Architecture summary . . . 2
1.2 Memory map . . . 2
1.3 VERA chip select . . . 3

2 Prerequisites **3**
2.1 FPGA toolchain . . . 3
2.1.1 Installing OSS CAD Suite (Recommended for Debian 11) . . . 3
2.1.2 Alternative: Manual installation via apt (Debian 12+) . . . 3
2.2 Flash programming tools . . . 4
2.2.1 Building openFPGALoader from source . . . 4
2.2.2 Iceprog . . . 4
2.3 Tool roles at a glance . . . 4

3 Build Flow **4**
3.1 Step 1 – Synthesis . . . 4
3.2 Step 2 – Place & Route . . . 5
3.3 Step 3 – Pack the bitstream . . . 5
3.4 Step 4 – Program the SPI flash . . . 5
3.4.1 Using openFPGALoader (recommended) . . . 6
3.4.2 Using iceprog (FTDI FT2232H) . . . 6
3.5 One-command build and flash . . . 6

4 Programmer Wiring **6**

5 Reset Sequencing **7**
5.1 Boot sequence . . . 7
5.2 Logic . . . 7
5.3 Circuit – 74LVC1G08 (SOT-23-5) . . . 7
5.4 Bill of materials . . . 8

6 Pin Assignment **8**
6.1 Complete pin table . . . 9

7 GeoRAM **10**
7.1 External SRAM wiring . . . 10
7.2 Address formula . . . 10
7.3 Data transceiver . . . 10

8 Timing **10**
8.1 Register writes . . . 11
8.2 SRAM read cycle . . . 11
8.3 SRAM write cycle . . . 11

9 Source File Reference **11**

1 Overview

This document describes the complete workflow for building and programming the FPGA firmware for the VERA RBL-C64 module. The firmware implements the Commodore 64 expansion-port bus interface on a Lattice iCE40UP5K. It drives the VERA chip select for the registers in `IO2` and implements a **GeoRAM 256 KB** on an external AS6C4008 SRAM, with the GeoRAM bank registers inside the FPGA.

There is a single bitstream (`C6502_Interface.bin`); no generics or build variants have to be chosen.

1.1 Architecture summary

- **Qualification by `/IO1` and `/IO2`**: every access is qualified by the C64 `/IO1` (port pin 7, `$DE00`–`$DEFF`) and `/IO2` (port pin 10, `$DF00`–`$DFFF`) lines. The C64 PLA asserts them only for CPU accesses (`AEC` high), so VIC-II fetches, whose addresses also appear on the port, never reach VERA or the SRAM. `/IOx` already decodes A15–A8, so only **A7–A0** are wired to the FPGA.
- **VERA registers** (`$DF00`–`$DF1F`, 32 B): the FPGA asserts `VERA_CS` for one bus cycle only when `/IO2` is low, A7–A5 are zero, the VERA select latch at `$DFFD` is set and PHI2 is high. The VERA FPGA drives the data bus.
- **VERA select latch** (`$DFFD`): a write of any non-zero value selects the VERA, `$00` deselects it. The latch always responds and selects only the VERA.
- **GeoRAM** (`$DE00`–`$DEFF` window, 256 KB total): served by an external 512K×8 SRAM (AS6C4008-55). Window and page/block registers (`$DFFE`/`$DFFF`) are **always active**, independent of the `$DFFD` latch, like the original cartridge: existing GeoRAM software works without touching `$DFFD`.
- **PHI2 synchronisation**: PHI2 is treated as a data input and synchronised to the 25 MHz FPGA clock via a two-stage flip-flop chain, producing a single-cycle `phi2_rise` pulse. Register writes are sampled `SAMPLE_CLKS` clocks later (Section 8).
- **No BRAM**: the design uses 0 of the 30 EBR blocks.

1.2 Memory map

Address	Size	Function	Implementation
`$DE00`–`$DEFF`	256 B	GeoRAM data window (one page), always active	External SRAM
`$DF00`–`$DF1F`	32 B	VERA registers, `VERA_CS` when the VERA is selected	VERA FPGA
`$DFFD`	1 B	VERA select latch (write only): ≠ `$00` selects, `$00` deselects	Register
`$DFFE`	1 B	GeoRAM page register (write only, 0–63, D5–D0)	Register
`$DFFF`	1 B	GeoRAM block register (write only, 0–15, D3–D0)	Register

`$DF20`–`$DFFC` are not decoded. The GeoRAM registers are decoded exactly at `$DFFE`/`$DFFF`, without the `$DF80`–`$DFFD` mirrors of some GeoRAM cartridges, so the latch at `$DFFD` does not collide with them. Latch and GeoRAM registers power up at zero (VERA deselected) and are not cleared by a C64 reset, like the original GeoRAM.

```
LDA #$04 : STA $DFFF   ; GeoRAM block 4 (always active)
LDA #$23 : STA $DFFE   ; GeoRAM page 35
```

```
LDA $DE45                ; GeoRAM byte $12345
LDA #$01 : STA $DFFD     ; select the VERA at $DF00-$DF1F
LDA #$00 : STA $DFFD     ; deselect the VERA
```

Listing 1: Typical use from 6510 code

1.3 VERA chip select

```
sel_vera <= '1' when IO2_N='0' and A(7 downto 5) = "000" and vera_sel
   ='1' else '0';
VERA_CS  <= '0' when sel_vera='1' and PHI2='1' else '1';
```

Listing 2: VERA decode in `C6502_Interface.vhd`

`VERA_CS` is never asserted for `$DF20`–`$DFFF` (latch and GeoRAM registers included), for `$DExx`, for any other address, for VIC-II cycles (no `/IO2`), or for `$DF00`–`$DF1F` while `$DFFD` holds zero. PHI2 gating hides the address decode glitches of PHI1.

2 Prerequisites

2.1 FPGA toolchain

While some tools are available via `apt` on Debian/Ubuntu, they are often outdated (especially on Debian 11). The recommended way to set up the toolchain is using the **OSS CAD Suite**, which provides pre-compiled, up-to-date versions of Yosys, nextpnr, GHDL, and all required plugins.

2.1.1 Installing OSS CAD Suite (Recommended for Debian 11)

1. Download the latest Linux x64 archive from:
   `https://github.com/YosysHQ/oss-cad-suite-build/releases/latest`
2. Extract the archive (e.g., to `$HOME/oss-cad-suite`).
3. Add the `bin` directory to your `PATH`:

```
export PATH="$HOME/oss-cad-suite/bin:$PATH"
```

This suite includes `yosys`, `nextpnr-ice40`, `icepack`, `iceprog`, and `ghdl` with the `ghdl-yosys-plugin` already integrated.

2.1.2 Alternative: Manual installation via apt (Debian 12+)

On newer distributions, you can try:

```
sudo apt install fpga-icestorm nextpnr-ice40 yosys ghdl
```

Note: if installing via `apt`, you will likely need to build the `ghdl-yosys-plugin` from source as described below, because it is rarely packaged.

```
git clone https://github.com/ghdl/ghdl-yosys-plugin
cd ghdl-yosys-plugin
make
sudo make install
```

2.2 Flash programming tools

On modern distributions (e.g. Debian 12+), `openFPGALoader` can be installed via package manager:

```
sudo apt install openFPGALoader
```

On older distributions like **Debian 11 (Bullseye)**, the package is not available and must be built from source.

2.2.1 Building openFPGALoader from source

First, install the required dependencies and build tools:

```
sudo apt install git cmake make g++ pkg-config libusb-1.0-0-dev \
                 libhidapi-dev libgpiod-dev libusbmuxd-dev
```

Then, clone the repository and build the project:

```
git clone https://github.com/trabucayre/openfpgaloader
cd openfpgaloader
mkdir build
cd build
# Use -DCMAKE_INSTALL_PREFIX=$HOME to install in your home directory
# or omit it to install system-wide (requires sudo make install)
cmake .. -DCMAKE_INSTALL_PREFIX=$HOME
make -j$(nproc)
make install
```

Ensure `$HOME/bin` is in your `PATH` if you used the home directory prefix.

2.2.2 Iceprog

The `iceprog` tool is already included in the `fpga-icestorm` package and supports FTDI-based programmers (e.g. FT2232H).

2.3 Tool roles at a glance

Tool	Input	Output	Purpose
`ghdl` (plugin)	VHDL sources	RTLIL	VHDL elaboration
`yosys`	RTLIL	`.json`	Synthesis
`nextpnr-ice40`	`.json + .pcf`	`.asc`	Place & route
`icepack`	`.asc`	`.bin`	Bitstream packing
`openFPGALoader` / `iceprog`	`.bin`	SPI flash	Programming

3 Build Flow

The design is a single VHDL file with no ROM to pre-load, so the build starts directly from synthesis.

3.1 Step 1 – Synthesis

```
cd firmware/FPGA
yosys -m ghdl -p \
```

```
    "ghdl --std=08 C6502_Interface.vhd -e C6502_Interface; \
     synth_ice40 -top C6502_Interface -json C6502_Interface.json"
```

Listing 3: Synthesis with Yosys and the GHDL plugin

The `-m ghdl` flag loads the GHDL plugin into Yosys, which handles VHDL analysis and elaboration. The entity has one generic, `SAMPLE_CLKS` (default 6), that sets the register write sample point (Section 8); the default needs no override.

Last measured results (OSS CAD Suite 2025-10-08):

Resource	Used	Available
ICESTORM_LC	40	5280
ICESTORM_RAM (EBR)	0	30
SB_IO (user I/O)	35	39
ICESTORM_SPRAM	0	4
Fmax (CLK)	**89.54 MHz** post-route (target 25 MHz)	

These figures predate the change that made the GeoRAM independent of the `$DFFD` latch; the logic is slightly smaller now. Re-measure after the next build.

3.2 Step 2 – Place & Route

Place & route assigns logic cells and routes signals to physical iCE40 resources, guided by the pin constraints file.

```
nextpnr-ice40 --up5k --package sg48 \
    --json C6502_Interface.json \
    --pcf board-pin-mapping.pcf \
    --asc C6502_Interface.asc
```

Listing 4: Place & route with nextpnr-ice40

Important: all pin numbers in `board-pin-mapping.pcf` are placeholders. Edit the file to match your actual schematic before this step (see Section 6).

3.3 Step 3 – Pack the bitstream

`icepack` converts the ASCII intermediate format produced by nextpnr into the binary bitstream that is written to the SPI flash.

```
icepack C6502_Interface.asc C6502_Interface.bin
```

Listing 5: Pack the binary bitstream

The `.bin` file is what gets programmed into the SPI flash. Bitstream size is device-dependent (not design-dependent) on the iCE40: **≈104 KB** for the iCE40UP5K SG48.

3.4 Step 4 – Program the SPI flash

The iCE40UP5K loads its configuration from an external SPI NOR flash at power-up. The programmer writes the bitstream into that flash.

3.4.1 Using openFPGALoader (recommended)

```
openFPGALoader -b ice40_generic C6502_Interface.bin
```

Listing 6: Program with openFPGALoader

Select the programmer cable explicitly if needed:

```
openFPGALoader --cable ft2232      C6502_Interface.bin   # FTDI
    FT2232H
openFPGALoader --cable ft232       C6502_Interface.bin   # FTDI FT232H
openFPGALoader --cable tigard      C6502_Interface.bin   # Tigard
openFPGALoader --cable raspberrypi C6502_Interface.bin   # Raspberry
    Pi GPIO
```

3.4.2 Using iceprog (FTDI FT2232H)

```
iceprog C6502_Interface.bin

# Erase + write (useful if previous write was partial)
iceprog -e 1048576 C6502_Interface.bin
```

Listing 7: Program with iceprog

3.5 One-command build and flash

The `firmware/FPGA/Makefile` wraps all steps:

```
cd firmware/FPGA
make                  # synthesise + P&R + pack -> C6502_Interface.
    bin
make flash            # write the bitstream via openFPGALoader
make flash-iceprog    # write the bitstream via iceprog (FTDI FT2232H
    )
make doc              # regenerate this PDF
make clean            # remove all generated files
```

4 Programmer Wiring

The SPI flash is shared between the FPGA (configuration bus at boot) and the programmer (writes the bitstream). During programming, the FPGA is held in reset by asserting `CRESET_B` low.

Programmer signal	iCE40UP5K pin	SPI flash pin
MOSI	SPI_SI	DI
MISO	SPI_SO	DO
SCK	SPI_SCK	CLK
CS#	–	CS#
CRST#	CRESET_B	–
CDONE	CDONE	–

Recommended SPI flash: Winbond W25Q16 (2 MB) or W25Q32 (4 MB).

Supported programmers:

Hardware	Interface	Tool
FTDI FT232H breakout	SPI bitbang	`iceprog`, `openFPGALoader`
FTDI FT2232H breakout	MPSSE SPI	`iceprog`, `openFPGALoader`
Tigard	JTAG/SPI	`openFPGALoader`
Raspberry Pi (GPIO)	SPI bitbang	`openFPGALoader`
dirtyJTAG (STM32F103)	JTAG	`openFPGALoader`

5 Reset Sequencing

The VERA RBL-C64 module requires a defined power-on boot sequence: the Commodore 64 and the VERA graphics chip must remain in reset until all FPGA configurations have completed. This is implemented with a single 74LVC1G08 logic gate — no MCU or firmware is needed.

5.1 Boot sequence

1. **Power-on:** both `VERA_RESET#` and `C64_RESET#` are held low.
2. **FPGA ready** (`FPGA_CDONE` goes high, ≈50–100 ms after power-on): `VERA_RESET#` is released, allowing VERA to start loading its own bitstream from its SPI flash.
3. **VERA ready** (`VERA_CDONE` goes high, ≈200–500 ms later): `C64_RESET#` is released and the C64 boots normally.

The natural SPI flash loading times provide all necessary delays; no RC timer or additional delay circuit is required.

5.2 Logic

Signal	Boolean expression
`VERA_RESET#`	`FPGA_CDONE`
`C64_RESET#`	`FPGA_CDONE` AND `VERA_CDONE`

`FPGA_CDONE`	`VERA_CDONE`	`VERA_RST#`	`C64_RST#`	Phase
0	0	0	0	Power-on / FPGA loading
1	0	1	0	FPGA ready, VERA loading
1	1	1	1	System ready

5.3 Circuit – 74LVC1G08 (SOT-23-5)

A single **74LVC1G08** (2-input AND gate, SOT-23-5) implements `C64_RESET#`. `VERA_RESET#` connects directly from `FPGA_CDONE` through a 33 Ω series resistor.

```
VCC --[4k7]--+-------- FPGA_CDONE (open-drain)
             |
             +--[33R]- VERA_RESET#
             |
             +-- A -\
                      +--- 74LVC1G08 ---[33R]--- C64_RESET#
             +-- B -/
             |
VCC --[4k7]--+-------- VERA_CDONE (open-drain)
```

Both CDONE outputs are open-drain; they require an external pullup to VCC. The 33 Ω series resistors limit current if an external driver (e.g. a reset button on the C64 `/RESET` line, which

pulls it to GND) conflicts with the 74LVC output.

Why 74LVC and not 74HC? The 74LVC family has *5 V tolerant inputs* while operating at 3.3 V VCC. The CDONE signals are at 3.3 V; the C64 `/RESET` line belongs to a 5 V TTL system. The 74LVC output at 3.3 V is correctly recognised as HIGH by 5 V TTL logic ($V_{IH} = 2.0$ V). The 74HC family is not suitable for this mixed-voltage interface.

5.4 Bill of materials

Ref	Value	Package	Function
U1	74LVC1G08	SOT-23-5	2-input AND gate
R1	4.7 kΩ	0402	Pullup for FPGA_CDONE
R2	4.7 kΩ	0402	Pullup for VERA_CDONE
R3	33 Ω	0402	Series resistor on VERA_RESET#
R4	33 Ω	0402	Series resistor on C64_RESET#

Total cost under 0.30 EUR; no firmware required.

6 Pin Assignment

Edit `board-pin-mapping.pcf` and replace every pin number with the actual pad number from your schematic before running nextpnr-ice40.

The `CLK` signal must be assigned to a **Global Buffer (GB) input** pin. Valid GB pins on the iCE40UP5K SG48 package are: **20, 28, 31, 38, 41, 43, 48**. The placeholder in the current PCF (pin 35) is *not* a GB pin.

Valid user I/O pin numbers for SG48 (39 total):

```
2, 3, 4, 6, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
23, 25, 26, 27, 28, 31, 32, 34, 35, 36, 37, 38, 39,
40, 41, 42, 43, 44, 45, 46, 47, 48
```

Pins 1, 5, 7, 8, 22, 24, 29, 30, 33 are power/ground or reserved and **must not** appear in the PCF file — nextpnr rejects them with an error.

I/O budget: the design uses 35 of the 39 user I/O pins; 4 are free.

Group	Pins
Address A[7:0] (A15–A8 decoded by `/IO1`, `/IO2`)	8
Data D[7:0] (input only, shared net with SRAM DQ)	8
`PHI2`, `RNW`	2
`IO1_N`, `IO2_N`	2
`VERA_CS`	1
SRAM: `SRAM_PAGE[5:0]`, `SRAM_BLOCK[3:0]`, CE/OE/WE	13
`CLK` (25 MHz, GB pin)	1
Total	**35**

6.1 Complete pin table

Signal	Dir	PCF pin	C64 port	Std	Description
CLK	In	35*	—	LVCMOS33	25 MHz system clock. Must land on a GB pin.
A[0]	In	40*	Y	LVCMOS33	Address bit 0. Also wired to SRAM A[0] (same for A1–A7).
A[1]	In	41*	X	LVCMOS33	Address bit 1
A[2]	In	42*	W	LVCMOS33	Address bit 2
A[3]	In	43*	V	LVCMOS33	Address bit 3
A[4]	In	44*	U	LVCMOS33	Address bit 4
A[5]	In	45*	T	LVCMOS33	Address bit 5
A[6]	In	46*	S	LVCMOS33	Address bit 6
A[7]	In	47*	R	LVCMOS33	Address bit 7
D[0]	In	11*	21	LVCMOS33	Data bit 0 (register writes). Shared net with SRAM DQ[0] (same for D1–D7).
D[1]	In	12*	20	LVCMOS33	Data bit 1
D[2]	In	13*	19	LVCMOS33	Data bit 2
D[3]	In	14*	18	LVCMOS33	Data bit 3
D[4]	In	15*	17	LVCMOS33	Data bit 4
D[5]	In	16*	16	LVCMOS33	Data bit 5
D[6]	In	17*	15	LVCMOS33	Data bit 6
D[7]	In	18*	14	LVCMOS33	Data bit 7
PHI2	In	34*	E	LVCMOS33	Phase-2 clock. Synchronised internally; not used as FPGA clock.
RNW	In	36*	5	LVCMOS33	Read/Not-Write. High = read, low = write.
IO1_N	In	26*	7	LVCMOS33	/IO1: CPU access to $DE00–$DEFF (active low).
IO2_N	In	27*	10	LVCMOS33	/IO2: CPU access to $DF00–$DFFF (active low).
VERA_CS	Out	32*	—	LVCMOS33	VERA chip select (active low), per cycle: /IO2 low, $DF00–$DF1F, VERA selected ($DFFD), PHI2 high.
SRAM_PAGE[0]	Out	2*	—	LVCMOS33	SRAM A[8], page bit 0
SRAM_PAGE[1]	Out	3*	—	LVCMOS33	SRAM A[9], page bit 1
SRAM_PAGE[2]	Out	4*	—	LVCMOS33	SRAM A[10], page bit 2
SRAM_PAGE[3]	Out	6*	—	LVCMOS33	SRAM A[11], page bit 3
SRAM_PAGE[4]	Out	9*	—	LVCMOS33	SRAM A[12], page bit 4
SRAM_PAGE[5]	Out	10*	—	LVCMOS33	SRAM A[13], page bit 5
SRAM_BLOCK[0]	Out	19*	—	LVCMOS33	SRAM A[14], block bit 0
SRAM_BLOCK[1]	Out	20*	—	LVCMOS33	SRAM A[15], block bit 1
SRAM_BLOCK[2]	Out	21*	—	LVCMOS33	SRAM A[16], block bit 2
SRAM_BLOCK[3]	Out	38*	—	LVCMOS33	SRAM A[17], block bit 3
SRAM_CE_N	Out	23*	—	LVCMOS33	SRAM chip enable (active low): /IO1 low and PHI2 high. Also the GeoRAM term of the data transceiver output enable.
SRAM_OE_N	Out	39*	—	LVCMOS33	SRAM output enable (active low), read cycles only.
SRAM_WE_N	Out	25*	—	LVCMOS33	SRAM write enable (active low), write cycles only.

** Pin numbers are placeholders. Replace with actual pad numbers from your board schematic.*

7 GeoRAM

7.1 External SRAM wiring

AS6C4008-55 (Alliance Memory, 512K×8, 55 ns, 2.7–5.5 V), lower 256 KB used. The SRAM sits on the 3.3 V side of the level shifters, on the same nets as the FPGA, and is powered at 3.3 V.

SRAM pin	Source	Note
A[7:0]	C64 A[7:0] (PCB net)	Direct trace, no FPGA I/O pin
A[13:8]	FPGA `SRAM_PAGE[5:0]`	Page register `$DFFE`
A[17:14]	FPGA `SRAM_BLOCK[3:0]`	Block register `$DFFF`
A[18]	GND (PCB)	Lower 256 KB of the 512 KB chip
DQ[7:0]	C64 D[7:0] (PCB net)	Shared with FPGA D[7:0]
CE#	FPGA `SRAM_CE_N`	
OE#	FPGA `SRAM_OE_N`	
WE#	FPGA `SRAM_WE_N`	

7.2 Address formula

16 blocks of 16 KB, each made of 64 pages of 256 bytes:

$$\text{offset} = \text{block} \times 16384 + \text{page} \times 256 + A[7:0]$$

```
sel_page   <= '1' when IO2_N='0' and A = x"FE" else '0';
sel_block  <= '1' when IO2_N='0' and A = x"FF" else '0';
sel_georam <= '1' when IO1_N='0' else '0';
```

Listing 8: GeoRAM decode in `C6502_Interface.vhd`

None of these terms depends on the `$DFFD` latch: the GeoRAM is always active.

7.3 Data transceiver

The board must drive the C64 data bus during GeoRAM reads. `SRAM_CE_N` is the GeoRAM term of the data transceiver output enable:

$$\overline{OE} = \text{NAND}\big(\text{PHI2},\ \text{NAND}(\text{R/W},\ \text{AND}(\texttt{VERA_CS},\ \texttt{SRAM_CE_N}))\big)$$

Every `$DExx` read is driven from the SRAM (GeoRAM always active); `$DF00`–`$DF1F` reads are driven only while the VERA is selected.

8 Timing

The C64 runs at 0.985 MHz (PAL) or 1.023 MHz (NTSC); PHI2 is high for ≈490 ns.

8.1 Register writes

The 6510 drives write data during PHI2 high, up to ≈150 ns after the rising edge. The latch (`$DFFD`) and the GeoRAM registers (`$DFFE`/`$DFFF`) are sampled `SAMPLE_CLKS` clock cycles after the synchronised PHI2 rising edge: with the default of 6 clocks at 25 MHz this is 240 ns after `phi2_rise`, i.e. ≈280–320 ns after the real edge, well inside the PHI2-high window. A PHI2 pulse that ends before the sample point never samples.

8.2 SRAM read cycle

Event	Time from PHI2 rise	Note
`/IO1` asserted by the PLA	≈50–80 ns	
`SRAM_CE_N` asserted	≈50–80 ns	`/IO1` AND PHI2
SRAM data valid (tAA = 55 ns)	≈105–135 ns	AS6C4008-55
6510 samples the data bus	≈490 ns	At PHI2 fall

The read margin is several hundred nanoseconds.

8.3 SRAM write cycle

`SRAM_WE_N` is asserted combinationally while `/IO1` is low, PHI2 is high and R/W is low. The pulse lasts ≈400 ns, far above the 40 ns tWP of the AS6C4008-55. The SRAM latches the data on the WE# rising edge (PHI2 fall); tDH = 0 ns is covered by the 6510 data hold after PHI2 fall.

`SRAM_PAGE`/`SRAM_BLOCK` change only on `$DFFE`/`$DFFF` writes, which are `/IO2` cycles, so they never change while CE# is low.

9 Source File Reference

File	Description
`C6502_Interface.vhd`	Top-level VHDL entity (Rev 6, C64 expansion port, VERA + GeoRAM). Contains the whole bus logic: `/IO1`/`/IO2` decode, VERA select latch, GeoRAM registers and SRAM control.
`board-pin-mapping.pcf`	Pin constraints for iCE40UP5K SG48. All pin numbers are placeholders – edit before use.
`Makefile`	Targets: `all` (default), `flash`, `flash-iceprog`, `doc`, `clean`.